

Monitoring and Observability Strategies for QEMU Processes in OpenStack on Kubernetes (OpenStack-Helm)

Joon-Seok Kim¹

¹Dept. of Software Science, Soongsil University
kimjoon0605@naver.com

Abstract

OpenStack-Helm (OSH) deploys OpenStack as Kubernetes Pods for operational convenience, but because the QEMU process is placed in the host's machine cgroup, standard Kubernetes monitoring tools such as kubelet and cAdvisor cannot observe the resource usage or contention state of the VM. In addition, the existing libvirt-based monitoring in OpenStack does not effectively expose real-time resource contention. To address this, we propose a read-only collector (qemu-exporter) that combines the libvirt socket with the host's cgroup/procfs to simultaneously extract resource usage and kernel-level contention metrics (PSI). Through real measurements, we verify the instrumentation gap of the standard Kubernetes monitoring stack, and confirm that the proposed collector accurately observes VM performance and resource-contention state with a low overhead of about 0.11% of one node CPU core.

Keywords: Kubernetes, OpenStack-Helm, QEMU, cgroup, resource monitoring, PSI

1. Introduction

Driven by Kubernetes' application-management capabilities and operational advantages, its adoption continues to grow worldwide [1]. In response, the OpenStack ecosystem increasingly deploys and manages OpenStack components as Kubernetes Pods through OpenStack-Helm (OSH). When a virtual machine (VM) is created in an OSH environment, the Nova component runs a QEMU process via the libvirtd inside the libvirt Pod through nova-compute. However, the resulting QEMU process is placed not in the Pod's cgroup region (kubepods.slice) but in the host's root machine (or machine.slice) cgroup [2]. As a result, standard Kubernetes monitoring tools cannot recognize the VM's resource usage, and the existing libvirt-API-based monitoring in OpenStack also fails to capture the real-time contention state. This work therefore implements a read-only collector that observes real-time resource contention without modifying the source code of OSH, libvirt, or Nova, and validates its low system overhead, presenting a practical way to close the observability gap of existing monitoring systems.

2. Background

2.1 OpenStack on Kubernetes (OpenStack-Helm)

OpenStack on Kubernetes is an architecture that operates the control plane and major components of OpenStack as Kubernetes Pods. By leveraging core Kubernetes features such as declarative state management and high availability, it improves the deployment and service-management efficiency of OpenStack. A representative framework implementing this deployment model is OpenStack-Helm (OSH).

2.2 libvirt statistics monitoring (virsh domstats)

The statistics provided by libvirt allow collecting metrics such as a VM's CPU usage time, network traffic, and storage I/O volume and rate. However, these statistics indicate only raw resource usage and do not provide contention metrics, i.e., the delay caused by resource shortage [3]. libvirt provides the vcpu.N.wait field to check contention, but this is only a fragmentary cumulative wait time and does not support the kernel's PSI (Pressure Stall Information) specification. It is therefore difficult to observe contention by

directly integrating with an external monitoring pipeline.

2.3 Kubernetes standard monitoring (kubelet/cAdvisor)

cAdvisor, embedded in the kubelet to handle monitoring in Kubernetes, collects only the kubepods cgroup hierarchy and does not traverse the host's machine cgroup hierarchy [4]. Because in an OSH environment the QEMU process is placed under the machine cgroup, it is excluded from cAdvisor's collection targets, producing an observability gap in which VM-related monitoring data fundamentally does not exist.

3. Method

3.1 qemu-exporter monitoring agent

This work proposes qemu-exporter, a read-only collector that, without modifying the source code of OSH, libvirt, Nova, or Kubernetes, reads the host's libvirt socket and cgroup/procfs file systems and exposes them in the Prometheus format. The agent operates in three stages. First, the identification stage: using the libvirt read-only socket, it obtains the PID of a running VM and, from the host's /proc/<PID>/cgroup path, finds the cgroup path for measurement through a domain-thread search algorithm. Second, the collection stage: from the obtained cgroup path it measures CPU usage time, memory usage

(memory.current), and the CPU PSI data (cpu.pressure), and from procfs (schedstat) it measures the per-thread CPU runqueue wait time. Third, the exposure stage: it attaches Nova metadata (instance ID, name, instance type, etc.) and a node-identity label to the collected VM resource-usage and contention metrics, and exposes them through the standard endpoint (/metrics) that a Prometheus server can scrape. Prometheus then requests (scrapes) /metrics to query the information. The collector is implemented in Go for fast execution and minimal host compute overhead, and it is built as a container image and deployed as a Pod for affinity with Kubernetes-based infrastructure.

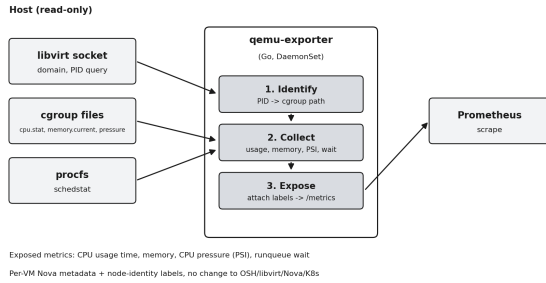


Fig. 1 qemu-exporter architecture

3.2 Experimental setup

The experiment was built on an AWS EC2 instance m5.2xlarge (8 vCPU, 32 GiB memory) running Ubuntu 24.04.4 LTS. OpenStack-Helm (OSH) was used to deploy the OpenStack control plane and major components as Kubernetes Pods, and two observation-target VMs were created through OpenStack before the real measurements were carried out.

3.3 Experiments and results

Three experiments were conducted to verify the necessity and effectiveness of qemu-exporter. The first experiment verified, through kubelet and cAdvisor, whether the VM's memory usage is omitted from the Kubernetes monitoring system.

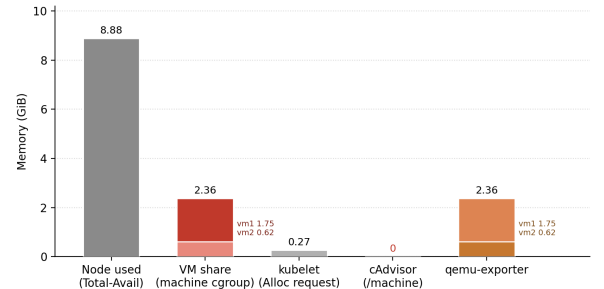


Fig. 2 Memory-usage visibility: kubelet/cAdvisor vs. qemu-exporter

In the experimental environment, the VM memory was deployed to occupy 27% (2.36 GiB) of the node's actually-used memory, yet the memory request recognized by kubelet was only 272 MiB, and no machine-cgroup-related data existed in cAdvisor. In contrast, qemu-exporter accurately measured each VM's memory usage together with Nova metadata (instance ID, name, instance type, etc.).

The second experiment injected a CPU-contention situation and comparatively analyzed whether the existing libvirt monitoring and the proposed qemu-exporter can each measure the contention. Under a scenario of 60 s before load, 120 s of injected CPU load, and 60 s after load, the CPU utilization and PSI data of each tool were collected at 2-second intervals.

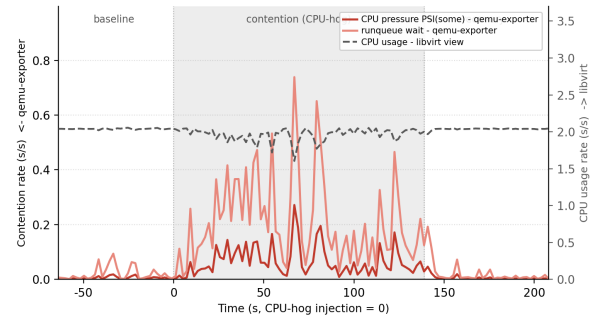


Fig. 3 CPU contention visibility: libvirt vs. qemu-exporter

By parsing the PSI(some) metric, which represents the fraction of time threads in the cgroup fail to be scheduled on a CPU, and the schedstat (runqueue wait) metric, which represents the sum of per-thread wait times, qemu-exporter

clearly captured the CPU-starvation stall state when load occurred. Specifically, under load the PSI(some) metric rose 12.5x (0.0117 -> 0.1467) and the runqueue-wait metric rose 10x (0.0454 -> 0.4631). In contrast, libvirt monitoring does not reflect the wait state when aggregating CPU utilization [3]; even as the load persisted, it measured only a 3% decrease in CPU utilization.

The third experiment measured the system overhead that qemu-exporter imposes on the host when operating as a monitoring agent.

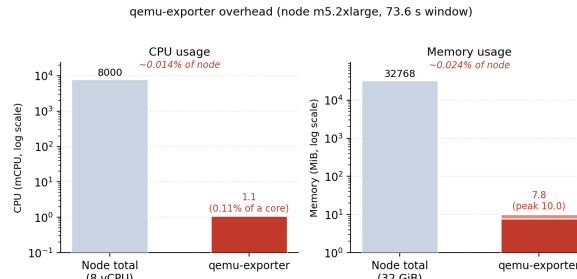


Fig. 4 Host compute resources consumed by qemu-exporter

The measurement showed that the CPU usage occupied across the whole node was about 0.11% of one core, and memory usage remained at about 10 MiB even at peak, confirming that the impact on host compute resources is negligible.

Table 1 summarizes and compares the three experimental results.

Axis	libvirt (virsh domstats)	cAdvisor/kubelet	qemu-exporter (proposed)
Resource usage (CPU, memory)	○	×	○
Resource contention (pressure, wait)	—	×	○
Nova metadata (uuid, flavor, ...)	○	×	○
Node Identity	×	○	○

○ provided — partial × not provided

Table 1 Monitoring coverage of libvirt, kubelet, and qemu-exporter

The default libvirt-based monitoring in OpenStack provides CPU and memory resource usage and Nova metadata. However, for contention information it provides only fragmentary cumulative wait time such as vcpu.N.wait, does not support the kernel-level PSI (Pressure Stall Information) specification, and also does not provide the node (physical-server) identity label needed to integrate with external monitoring pipelines such as Prometheus or OpenTelemetry, making it hard to observe real-time contention. In addition, the cAdvisor-based kubelet used for in-node Pod monitoring cannot collect the VM at all because the VM does not belong to the cgroup region it governs. In contrast, the proposed qemu-exporter integrates resource-contention metrics obtained by parsing cgroup/procfs on top of the information provided by existing libvirt, and we verified that it exposes them in a format that integrates with external monitoring tools.

4. Conclusion

This work empirically verified the monitoring gap that arises because, in an OSH environment, the QEMU process is placed not in Kubernetes' kubepods but in the host's machine cgroup region. The cAdvisor-based kubelet could not recognize VM resources and underestimated node memory usage by 27%, and libvirt-based monitoring, even

though PSI(some) and runqueue-wait metrics surged 12.5x and 10x respectively during contention, measured only a 3% decrease in CPU usage and thus failed to effectively capture the contention state. To close this 'observability-path gap', we proposed qemu-exporter, which reads host kernel data in a read-only manner. Through it, VM resource usage and contention metrics are collected without omission, and a practical foundation is established for observing the resource state of VMs and OSH components together at a single point with low overhead. However, this experiment was performed on AWS EC2 with pure QEMU emulation without KVM acceleration, so the guest-internal CPU steal metric could not be confirmed. Reproducing this in a KVM environment to check whether guest-side contention metrics can be captured remains future work.

References

- [1] Cloud Native Computing Foundation, "Kubernetes Established as the De Facto 'Operating System' for AI as Production Use Hits 82% in 2025 CNCF Annual Cloud Native Survey," CNCF, 2026. <https://www.cncf.io/announcements/2026/01/20/kubernetes-established-as-the-de-facto-operating-system-for-ai-as-production-use-hits-82-in-2025-cncf-annual-cloud-native-survey/> (accessed Sep. 2026)
- [2] libvirt, "Control Groups Resource Management," libvirt.org, 2026. <https://libvirt.org/cgroups.html> (accessed Sep. 2026)
- [3] libvirt, "virsh(1) Manual — domstats," libvirt.org, 2026. <https://libvirt.org/manpages/virsh.html#domstats> (accessed Sep. 2026)
- [4] Kubernetes, "Tools for Monitoring Resources," Kubernetes Documentation, 2026. <https://kubernetes.io/docs/concepts/cluster-administration/resource-usage-monitoring/> (accessed Sep. 2026)